



VIT[®]
BHOPAL

VITYARTHI PROJECT

THE E-CHALLAN & TRAFFIC INCIDENT LOGGING SYSTEM

NAME: AYUSH SHARMA

Regn. No.: 25BAI10371

FACULTY: Jay Prakash Maurya

SLOT: F11+F12

1. Introduction

1.1 Objective

The objective of this project is to design and implement a robust, enterprise-grade backend system for an E-Challan and Traffic Incident Logger. Built using Object-Oriented Java, the system facilitates real-time logging of traffic offenses, automated fine calculation, and secure receipt generation.

1.2 Scope

This phase of the project focuses strictly on the core backend architecture, including relational database management, CRUD operations, and offline data persistence. By establishing a secure and crash-proof command-line interface (CLI) and database schema, this system serves as the foundational data layer for future integration with automated Computer Vision and Automated License Plate Recognition (ALPR) models.

2. Problem Statement

Traditional traffic monitoring relies heavily on manual data entry and paper-based challan systems, which are prone to human error, data loss, and inefficiency. As urban traffic volume increases, there is a critical need for a digitized, automated system capable of handling high-speed transactional data. The problem addressed in this phase is the lack of a secure, crash-proof database architecture and backend system required to safely store and process incident data before integrating live computer vision cameras.

3. FUNCTIONAL REQUIREMENTS

FR-1 (Vehicle Registration): The system must allow administrators to insert new vehicle records (License Plate, Owner Name, Contact, Model) into the database.

FR-2 (Incident Logging): The system must record traffic violations, linking a specific vehicle to a predefined offense type using Foreign Keys.

FR-3 (E-Challan Generation): The system must query the database to retrieve unpaid incidents, calculate fines, and display a formatted receipt.

FR-4 (Offline Backups): The system must automatically generate physical .txt files of every challan on the local hard drive.

FR-5 (Backup Retrieval): The system must be able to list and read offline backup files directly from the terminal.

4. NON-FUNCTIONAL REQUIREMENTS

NFR-1 (Security): The system must prevent SQL injection attacks by using pre-compiled, parameterized queries for all database inputs.

NFR-2 (Data Integrity): The database must reject orphaned records (e.g., logging an incident for an unregistered vehicle) via strict constraint enforcement.

NFR-3 (Reliability): The system must implement dual-persistence (MySQL + Local File I/O) so that receipts are not lost during database downtime.

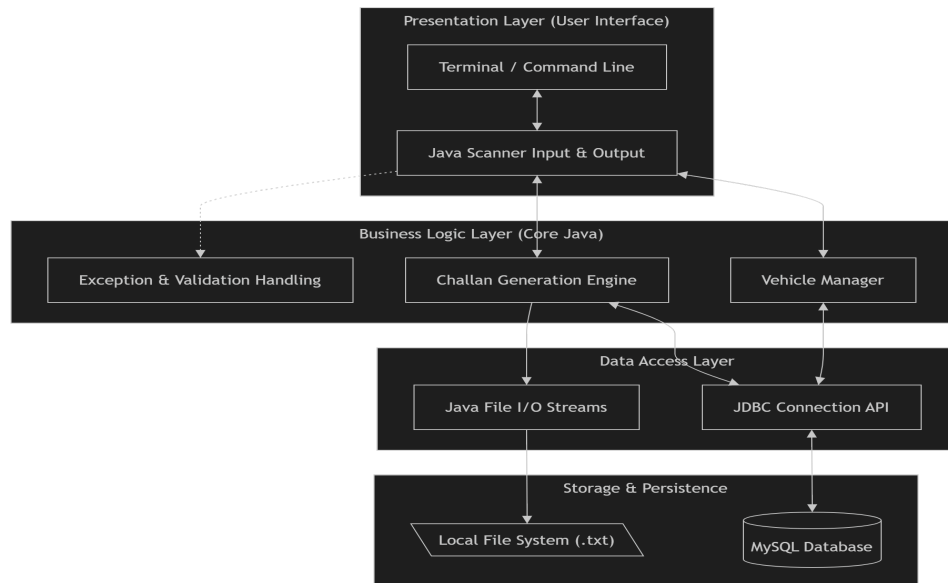
NFR-4 (Maintainability): The codebase must strictly adhere to Object-Oriented Programming principles (encapsulation, modularity) to allow future integration with Python/OpenCV modules.

5. Technology Stack

- **Programming Language:** Java (JDK 24)
- **Database Management:** MySQL
- **Database Connectivity:** Java Database Connectivity (JDBC) via MySQL Connector/J
- **Data Persistence:** Java I/O Streams (`BufferedReader`, `BufferedWriter`, `FileWriter`)
- **Version Control:** Git & GitHub
-

6. System Architecture

The E-Challan logging system is designed using a modular, 3-tier architecture to ensure a strict separation of concerns between the user interface, backend processing, and data storage. **Figure 1** below illustrates the high-level data flow and component interaction across the Java application and the MySQL server.

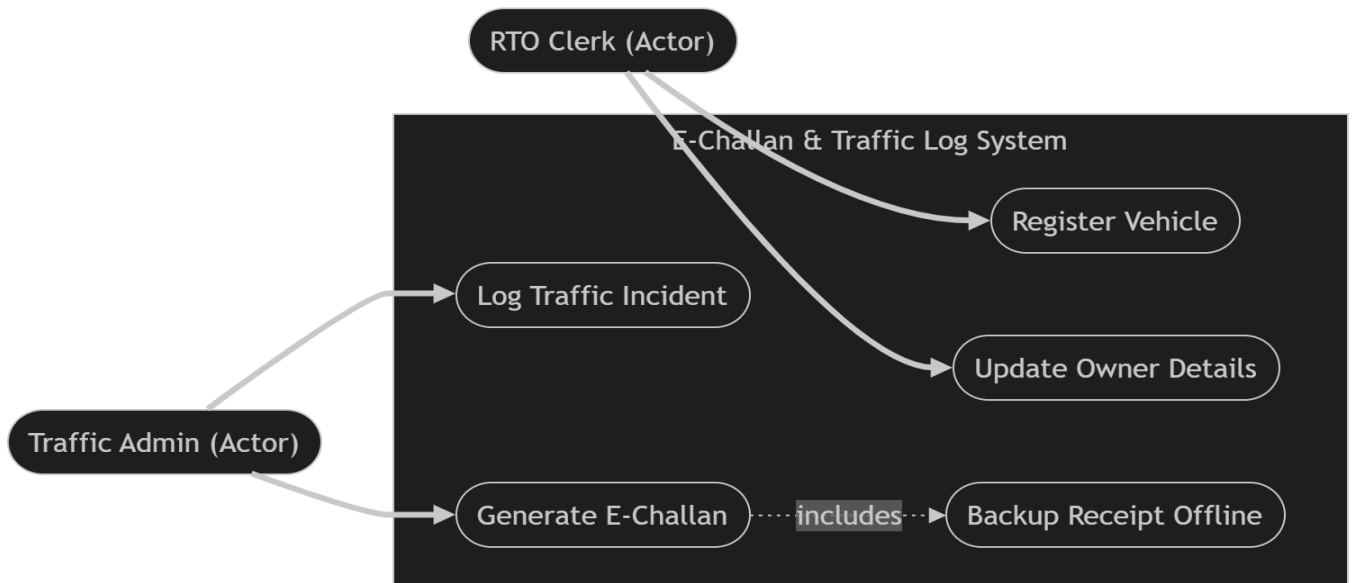


As demonstrated in the architecture diagram above, the system executes across three primary layers:

- **Presentation Layer (CLI):** A Java-based command-line interface handles all user inputs. It routes terminal commands through a central `MainMenu` directly to specific utility classes (`VehicleManager`, `IncidentManager`, `ChallanGenerator`).
- **Data Access Layer (JDBC):** The application communicates with the MySQL server using the `mysql-connector-java.jar` driver. It uses parameterized `PreparedStatement` queries to safely pass data from the Java runtime into the database without risking SQL injection.
- **Persistence Layer:** The system features a dual-persistence model. Live transactional data is stored in the relational MySQL database, while physical `.txt` receipts are asynchronously written to the local file system using Java File I/O to ensure offline redundancy.

6.1 User Interaction Model

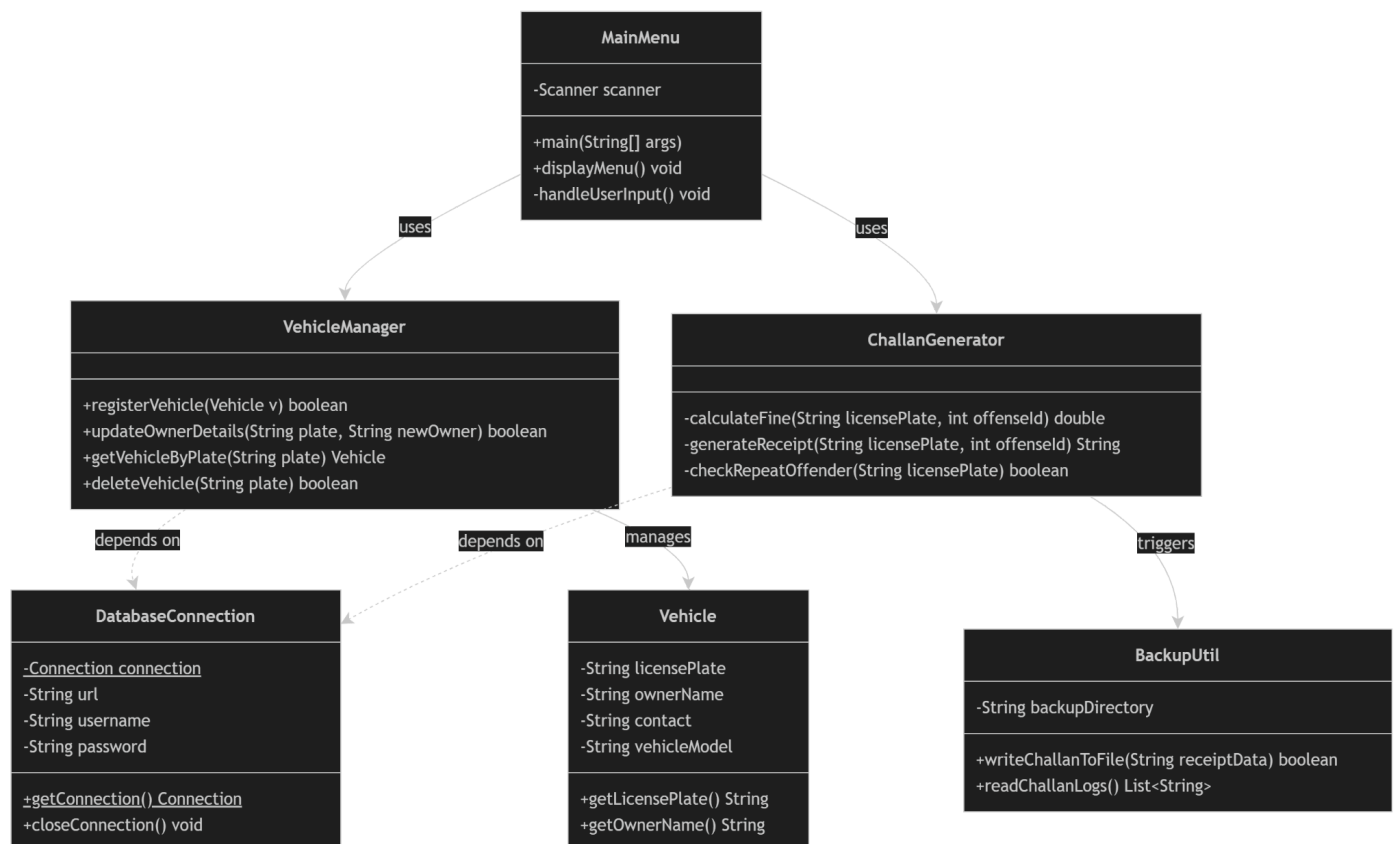
To map out the exact functional requirements and system boundaries, a Use Case model was established. **Figure 2** details the specific actions the system administrator can perform within the CLI environment.



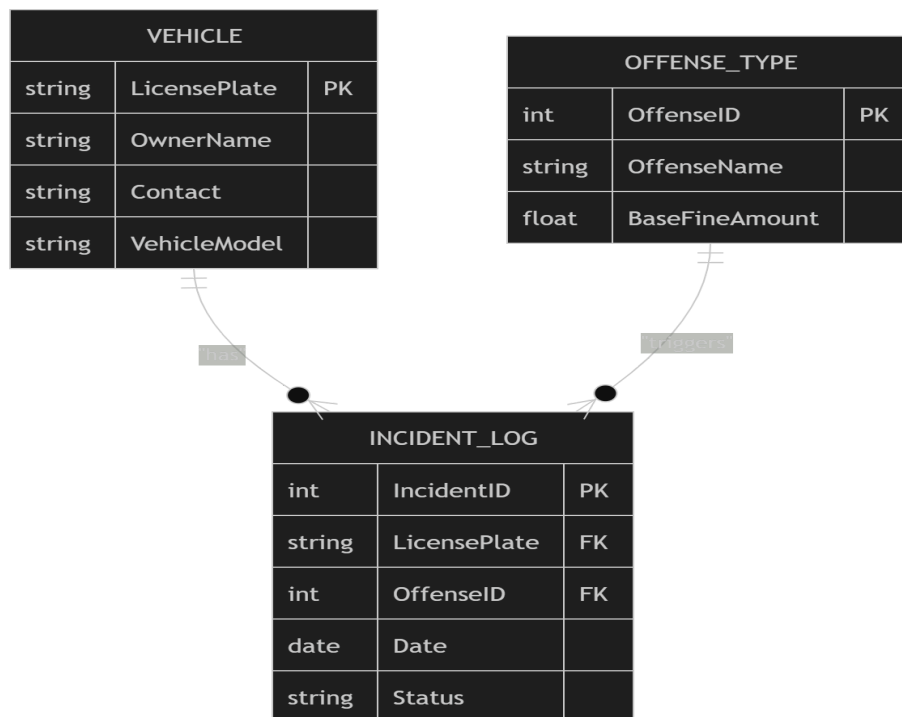
The administrative actor is granted full operational access to the four primary modules. The diagram highlights how the "Generate E-Challan" and "View Offline Backups" use cases are completely decoupled from the live database writing processes, ensuring that generating read-only reports does not lock the database tables during active incident logging.

6.2 Object-Oriented Class Structure

To maintain clean code and reusability, the Java backend is structured using strict Object-Oriented principles. **Figure 3** maps the core classes, their internal methods, and how they depend on the central `DatabaseConnection` and `BackupUtil` utilities to process data.



7. Database Design



The system relies on a normalized relational database ([echallan_db](#)) structured to maintain strict data integrity through Primary and Foreign Keys:

- **Vehicles Table:** Stores registered vehicle data. The [LicensePlate](#) serves as the Primary Key.
- **OffenseTypes Table:** A reference table storing standardized traffic violations (e.g., Speeding, No Helmet) and their base fines. [OffenseID](#) serves as the Primary Key.

- **IncidentLogs Table:** The central transaction table. It uses `LicensePlate` and `OffenseID` as Foreign Keys to link an offending vehicle to a specific violation. This prevents the system from logging an incident for an unregistered vehicle.

8. Core Implementation & Code Snippets

8.1 Database Connectivity (JDBC)

The `DatabaseConnection` class utilizes the Singleton pattern approach to establish a secure link between the Java runtime and the MySQL server.

```
public static Connection getConnection() {  
    Connection connection = null;  
    try {  
        connection =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/echall  
an_db", "root", "password");  
    } catch (SQLException e) {  
        System.out.println("[ERROR] Database Connection Failed!");  
    }  
    return connection;  
}
```

8.2 Secure CRUD Operations

To handle user inputs securely, the system utilizes **PreparedStatement**. This pre-compiles the SQL query, protecting the database from SQL injection attacks during data insertion.

```
public static void registerVehicle(String plate, String owner, String contact,
String model) {

    String query = "INSERT INTO Vehicles (LicensePlate, OwnerName,
Contact, VehicleModel) VALUES (?, ?, ?, ?)";

    try (Connection conn = DatabaseConnection.getConnection();

        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setString(1, plate);

        pstmt.setString(2, owner);

        // ... execution logic

    }

}
```

8.3 Advanced SQL Integration (JOINS)

The `ChallanGenerator` class demonstrates advanced SQL querying by joining three separate tables simultaneously to construct a complete, human-readable traffic ticket.

```
String query = "SELECT i.IncidentID, i.IncidentDate, v.OwnerName,  
o.OffenseName, o.BaseFineAmount " +  
    "FROM IncidentLogs i " +  
    "JOIN Vehicles v ON i.LicensePlate = v.LicensePlate " +  
    "JOIN OffenseTypes o ON i.OffenseID = o.OffenseID " +  
    "WHERE i.LicensePlate = ? AND i.Status = 'UNPAID'";
```

8.4 Offline Backups via File I/O

To fulfill redundancy requirements, the system leverages `BufferedWriter` and `FileWriter` to dynamically create directories and write physical `.txt` files to the local disk.

```
private static void saveReceiptToFile(String plate, int incidentId, String
receiptData) {

    File directory = new File("challans");

    if (!directory.exists()) directory.mkdir();

    String fileName = "challans/Challan_" + plate + "_" + incidentId + ".txt";

    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {

        writer.write(receiptData);

    } catch (IOException e) {

        System.out.println("[ERROR] Failed to save offline backup.");

    }

}
```

9. System Output & Testing

Test Case 1: Database Connection & Vehicle Registration

```
=====
  E-CHALLAN & TRAFFIC LOG SYSTEM - CLI
=====
[SYSTEM] Testing Database Connection...
[SYSTEM] Database Connected Successfully!
```

Main Menu:

1. Register a New Vehicle
2. Log a Traffic Incident
3. Generate E-Challan (Penalty)
4. View Offline Backups
5. Exit System

1. Register a New Vehicle
2. Log a Traffic Incident
3. Generate E-Challan (Penalty)
4. View Offline Backups
5. Exit System

Enter your choice (1-5): 1

--- REGISTER NEW VEHICLE ---

Enter License Plate (e.g., MP-43-XY-1234): RJ-14-AB-9876

Enter Owner Name: ayush

Enter Contact Number: 123456789

Enter Vehicle Model: ciaz

[SUCCESS] Vehicle registered successfully in the database!

Test Case 2: Logging an Incident

```
Main Menu:
1. Register a New Vehicle
2. Log a Traffic Incident
3. Generate E-Challan (Penalty)
4. View Offline Backups
5. Exit System

Enter your choice (1-5): 2

--- LOG TRAFFIC INCIDENT ---
Enter Offending License Plate: RJ-14-AB-9876

--- AVAILABLE OFFENSES ---
1. Speeding (Base Fine: Rs.1000.0)
2. Running Red Light (Base Fine: Rs.500.0)
3. No Helmet (Base Fine: Rs.1000.0)
4. Driving Without License (Base Fine: Rs.5000.0)

Enter Offense ID (1-4): 1

[SUCCESS] Traffic incident logged successfully for vehicle RJ-14-AB-9876!
```

Test Case 3: E-Challan Generation & File I/O

```
Main Menu:
1. Register a New Vehicle
2. Log a Traffic Incident
3. Generate E-Challan (Penalty)
4. View Offline Backups
5. Exit System

Enter your choice (1-5): 3

--- GENERATE E-CHALLAN ---
Enter License Plate to check unpaid challans: RJ-14-AB-9876

=====
                E-CHALLAN RECEIPT
=====
Incident ID : 3
Date       : 2026-09-14 16:30:06
Vehicle    : RJ-14-AB-9876
Owner Name : ayush
Offense    : Speeding
Total Fine : Rs. 1000.00
Status     : UNPAID
=====

[SUCCESS] Offline backup saved via File I/O to: challans/Challan_RJ-14-AB-9876_3.txt
```

10. Conclusion & Future Scope

10.1 Conclusion

This phase successfully delivered a highly modular, secure, and crash-proof backend architecture. The application successfully integrates Java Object-Oriented principles with a

relational MySQL database and local file handling. The strict enforcement of Foreign Keys ensures data integrity, while the JDBC implementation handles real-time data flow efficiently.

10.2 Future Scope & Integration

With the core backend and database schema now fully operational and version-controlled via GitHub, the project is ready for its final engineering phase. The manual CLI data-entry layer will be replaced by an automated pipeline. The remaining team members will deploy Python-based computer vision scripts (using OpenCV) to analyze live CCTV camera feeds, detect offending license plates via ALPR, and push that automated data directly into this Java backend for seamless, human-free e-challan generation.

11. Challenges Faced

- **Driver Configuration:** Properly importing and linking the `mysql-connector-java.jar` library within the IntelliJ IDEA environment required troubleshooting module dependencies.
- **Version Control Pathing:** Navigating nested project directories (`Project/database/`) via the PowerShell terminal caused synchronization errors during Git commits, which was resolved by utilizing IntelliJ's built-in VCS GUI.
- **Relational Constraints:** Managing data deletion carefully; attempting to delete a vehicle that already had an incident logged triggered foreign key violation errors, requiring careful management of table dependencies.

12. Learnings & Key Takeaways

- **Database Management:** Gained practical experience designing normalized relational schemas and writing complex SQL `JOIN` queries.
- **Security Practices:** Learned the critical importance of using `PreparedStatement` to sanitize inputs against SQL injection.
- **System Architecture:** Successfully implemented a 3-tier architecture, completely separating user interface logic from backend data processing.
- **Version Control:** Mastered pushing local code repositories to remote GitHub branches for collaborative development.

13. Source Code Repository

The complete source code, database schema (`schema.sql`), and version history for this project can be accessed at:

GitHub Repository:

<https://github.com/AyushSharma-AS/The-E-Challan-Traffic-Log-System>